

Algoritmi

Table of contents

Algoritmi	2
Ordinamento	2
SELECTION SORT	2
Idea dell'algoritmo	3
Esempio passo per passo	3
Calcolo della complessità computazionale	5
Numero di confronti	5
Esercizi	6
MERGE	7
Idea dell'algoritmo	7
Implementazione in Python	7
Esempio di utilizzo	8
Esecuzione passo-passo	8
Perché servono gli ultimi due cicli <code>while</code> ?	9
Versione più compatta con slicing	10
Complessità computazionale (tempo)	10
Ricerca	10
Ricerca lineare	10
Esempio	11
Implementazione in Python	11
Complessità computazionale della ricerca lineare	12
ESERCIZI:	12
Ricerca binaria	12
Idea dell'algoritmo	13
Esempio	13
Implementazione iterativa in Python	13
Perché si usa <code>//</code> ?	14
Complessità computazionale della ricerca binaria	15
Osservazione importante sul costo dell'ordinamento	15
Macchine a stati finiti (cenni)	16

Algoritmi

Ordinamento

In informatica, l'**ordinamento** è il processo con cui si dispone una collezione di dati secondo un criterio definito, ad esempio in ordine crescente o decrescente.

Può riguardare numeri, parole, record o altri elementi.

Esempi:

- 3, 1, 5, 2 → 1, 2, 3, 5 (ordine crescente)
 - pera, mela, banana → banana, mela, pera (ordine alfabetico)
-

Per illustrare gli algoritmi di ordinamento, per comodità consideriamo una lista di interi, ma gli stessi algoritmi si applicano a strutture dati più generali.

L'essenziale è che esista una **relazione d'ordine** tra gli elementi da ordinare.

L'elemento della lista può essere un dizionario. Occorre ordinare il dizionario secondo un criterio che coinvolge uno o più chiavi del dizionario.

Esempio:

```
d = { paga_oraria: 26,  
      ore_lavorate: 32,  
      # altri campi  
}
```

- Ordinare secondo la paga oraria crescente
- Ordinare secondo lo stipendio crescente (`ore_lavorate * paga_oraria`)

SELECTION SORT

Selection sort è un algoritmo di ordinamento semplice. Serve a ordinare una sequenza di valori, ad esempio una lista di numeri, dal più piccolo al più grande.

Idea base:

- ad ogni passo si cerca l'elemento più piccolo nella parte non ancora ordinata della lista e lo si sposta nella posizione corretta.

è l'occasione per ripassare:

- confronto tra elementi;
- scambio di valori;
- uso di cicli annidati;
- analisi della complessità computazionale.

Idea dell'algoritmo

Supponiamo di voler ordinare la lista:

```
[5, 2, 4, 1, 3]
```

Il selection sort divide idealmente la lista in due parti:

1. una parte iniziale già ordinata;
2. una parte finale ancora da ordinare.

All'inizio la parte ordinata è vuota. A ogni iterazione l'algoritmo cerca il minimo nella parte non ordinata e lo scambia con il primo elemento di quella parte.

Esempio passo per passo

Lista iniziale:

```
[5, 2, 4, 1, 3]
```

Primo passo: si cerca il minimo in tutta la lista. Il minimo è 1. Lo si scambia con il primo elemento, cioè 5.

```
[1, 2, 4, 5, 3]
```

Ora 1 è nella posizione corretta.

Secondo passo: si cerca il minimo nella parte non ordinata:

```
[2, 4, 5, 3]
```

Il minimo è 2, che è già nella posizione corretta. La lista resta:

```
[1, 2, 4, 5, 3]
```

Terzo passo: si cerca il minimo nella parte:

```
[4, 5, 3]
```

Il minimo è 3. Lo si scambia con 4.

```
[1, 2, 3, 5, 4]
```

Quarto passo: si cerca il minimo nella parte:

```
[5, 4]
```

Il minimo è 4. Lo si scambia con 5.

```
[1, 2, 3, 4, 5]
```

La lista è ordinata.

```
# Selection Sort

def selection_sort(lista):
    n = len(lista)

    for i in range(n):
        # miglioramento: range(n-1):
        # indice del valore minimo
        idx_minimo = i

        # cerca il minimo nella parte restante
        for j in range(i + 1, n):
            if lista[j] < lista[idx_minimo]:
                idx_minimo = j

        # scambia gli elementi

        tmp = lista[idx_minimo]
        lista[idx_minimo] = lista[i]
        lista[i] = tmp
        # miglioramento: scambia solo se diversi
        # if idx_minimo != i:...

    return lista
```

```
# Esempio
numeri = [64, 25, 12, 22, 11]

ordinata = selection_sort(numeri)

print("Lista ordinata:", ordinata)
```

Commenti:

- la funzione ordina **in place** ed in più rende anche la lista ordinata. Posso usarla così:

```
ordinata = selection_sort(numeri)
```

- se elimino il **return** ordinerà solo **in place**. Dovrò usarla così:

```
selection_sort(numeri)
```

- se voglio lasciare inalterata la lista iniziale devo effettuare una **deep copy**

Calcolo della complessità computazionale

Per calcolare la complessità del selection sort bisogna contare quante operazioni significative vengono eseguite al crescere della dimensione dell'input.

Indichiamo con **n** il numero di elementi della lista.

L'operazione più importante da contare è il **confronto**:

```
if lista[j] < lista[indice_minimo]:
```

Numero di confronti

Il ciclo esterno viene eseguito **n - 1** volte.

Alla prima iterazione, cioè quando **i = 0**, il ciclo interno confronta il primo elemento con tutti gli altri elementi della lista. Il numero di confronti è:

```
n - 1
```

Alla seconda iterazione, quando **i = 1**, la prima posizione è già ordinata. Il ciclo interno controlla solo gli elementi restanti. Il numero di confronti è:

$$n - 2$$

Alla terza iterazione il numero di confronti è:

$$n - 3$$

Si continua così fino all'ultimo confronto:

$$1$$

Il numero totale di confronti è quindi:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 1$$

Questa è la somma dei primi $n - 1$ numeri naturali:

$$1 + 2 + 3 + \dots + (n - 1)$$

La formula della somma è:

$$n(n - 1) / 2$$

Quindi il numero totale di confronti è:

$$n(n - 1) / 2$$

Quando n diventa molto grande, il termine dominante è n^2 . Le costanti e i termini di grado inferiore vengono ignorati nell'analisi asintotica.

Per questo motivo la complessità temporale del selection sort è $O(n^2)$

Esercizi

- Algoritmo **in place** che restituisce **None**
- Algoritmo che restituisce una lista ordinata ma che non altera la lista originale
- Algoritmo che permetta di ordinare in senso crescente o decrescente a seconda del valore di un parametro di controllo
- Algoritmo che ordina una lista di dizionari
- Algoritmo che ordina contemporaneamente DUE liste in base ai valori di una lista

MERGE

La funzione `merge` serve a **fondere due liste già ordinate** in una nuova lista ordinata.

Esempio:

```
lista_a = [1, 4, 7]
lista_b = [2, 3, 8]
```

La funzione `merge` deve produrre:

```
[1, 2, 3, 4, 7, 8]
```

Idea dell'algoritmo

Supponiamo di avere due liste già ordinate:

```
lista_a = [1, 4, 7]
lista_b = [2, 3, 8]
```

L'idea è usare due indici:

- `i`, che scorre la lista **sinistra**;
- `j`, che scorre la lista **destra**.

A ogni passo si confrontano gli elementi correnti:

```
lista_a[i]
lista_b[j]
```

L'elemento più piccolo viene aggiunto alla lista risultato. Poi si avanza l'indice della lista da cui è stato preso quell'elemento.

Implementazione in Python

```
def merge(lista_1, lista_2):
    lista_3 = []
    i1 = 0
    i2 = 0

    while i1 < len(lista_1) and i2 < len(lista_2):
        if lista_1[i1] <= lista_2[i2]:
```

```

        lista_3.append(lista_1[i1])
        i1 += 1
    else:
        lista_3.append(lista_2[i2])
        i2 += 1

    while i1 < len(lista_1):
        lista_3.append(lista_1[i1])
        i1 += 1

    while i2 < len(lista_2):
        lista_3.append(lista_2[i2])
        i2 += 1

    return lista_3

```

Esempio di utilizzo

```

lista_a = [1, 4, 7]
lista_b = [2, 3, 8]

merge(lista_a, lista_b)

```

Output:

```
[1, 2, 3, 4, 7, 8]
```

Esecuzione passo-passo

Consideriamo:

```

lista_a = [1, 4, 7]
lista_b = [2, 3, 8]

```

All'inizio:

```

lista_3 = []
i1 = 0
i2 = 0

```

La funzione confronta `lista_1[i1]` e `lista_2[i2]`.

Passo	<code>lista_1[i]</code>	<code>lista_2[j]</code>	Elemento scelto	<code>lista_3</code>
1	1	2	1	[1]
2	4	2	2	[1, 2]
3	4	3	3	[1, 2, 3]
4	4	8	4	[1, 2, 3, 4]
5	7	8	7	[1, 2, 3, 4, 7]

A questo punto la lista `lista_1` è terminata. Rimane solo l'elemento 8 nella lista `lista_2`, quindi viene copiato in `lista_3`.

Risultato finale:

```
[1, 2, 3, 4, 7, 8]
```

Perché servono gli ultimi due cicli `while`?

Il primo ciclo:

```
while i1 < len(lista_1) and i2 < len(lista_2):
```

continua finché entrambe le liste hanno ancora elementi da confrontare.

Quando una delle due liste termina, il ciclo si ferma. Tuttavia, l'altra lista potrebbe avere ancora elementi non copiati.

Per questo si usano due cicli finali:

```
while i1 < len(lista_1):
    risultato.append(lista_1[i1])
    i1 += 1
```

```
while i2 < len(lista_2):
    risultato.append(lista_2[i2])
    i2 += 1
```

Questi cicli copiano nel risultato gli elementi rimasti.

Poiché le due liste di partenza sono già ordinate, gli elementi rimasti possono essere copiati direttamente senza altri confronti.

Versione più compatta con slicing

In Python si può scrivere anche una versione più breve:

```
def merge(lista_1, lista_2):  
    #...  
  
    risultato.extend(lista_1[i1:])  
    risultato.extend(lista_2[i1:])  
  
    return lista_3
```

Questa versione è più compatta, ma didatticamente la versione con i tre cicli **while** è spesso più chiara, perché mostra esplicitamente tutti i passaggi.

Complessità computazionale (tempo)

Ogni elemento viene inserito nella lista `lista_3` una sola volta. In totale, ogni elemento viene processato una volta.

La complessità temporale è quindi $O(n_1 + n_2)$

Ricerca

- **ricerca lineare**;
- **ricerca binaria**.
- dato un valore da cercare, è presente nella lista? Se sì, in quale posizione?

La differenza principale è che la **ricerca lineare** funziona su qualunque lista, mentre la **ricerca binaria** richiede che la lista sia già ordinata.

Ricerca lineare

La ricerca lineare controlla gli elementi della lista uno alla volta, partendo dal primo.

L'algoritmo confronta ogni elemento con il valore cercato:

1. controlla il primo elemento;
2. se è uguale al valore cercato, restituisce la sua posizione;
3. altrimenti passa all'elemento successivo;
4. continua fino a trovare il valore oppure fino alla fine della lista.

Il nome **lineare** deriva dal fatto che, nel caso peggiore, l'algoritmo deve scorrere tutta la lista in modo sequenziale.

Esempio

Supponiamo di avere questa lista:

```
lista = [7, 3, 9, 2, 5]
```

Vogliamo cercare il valore 2.

L'algoritmo procede così:

Posizione	Valore controllato	Confronto
0	7	7 non è 2
1	3	3 non è 2
2	9	9 non è 2
3	2	trovato

Il valore 2 si trova in posizione 3.

Implementazione in Python

```
def ricerca_lineare(lista, valore):  
    for i in range(len(lista)):  
        if lista[i] == valore:  
            return i  
  
    return None
```

Esempio d'uso:

```
#| echo: true  
  
lista = [7, 3, 9, 2, 5]  
  
posizione = ricerca_lineare(lista, 2)  
print(posizione)
```

Output:

```
3
```

Complessità computazionale della ricerca lineare

Indichiamo con n il numero di elementi della lista.

La ricerca lineare controlla gli elementi uno alla volta. Il numero di confronti dipende dalla posizione del valore cercato.

- Caso migliore:

Il valore cercato si trova nella prima posizione. In questo caso basta un solo confronto. Complessità $O(1)$

- Caso peggiore:

Il caso peggiore si verifica quando il valore cercato è nell'ultima posizione oppure il valore non è presente nella lista.

In entrambi i casi l'algoritmo deve controllare tutti gli n elementi. Quindi la complessità è $O(n)$

- Caso medio

Nel caso medio, se il valore è presente e può trovarsi con uguale probabilità in qualunque posizione, l'algoritmo controlla circa metà lista. Il numero di operazioni da effettuare è proporzionale a $n/2$, quindi la complessità è $O(n)$.

ESERCIZI:

- cercare **tutti** gli elementi uguali a **target** -> la funzione deve rendere una lista di indici, oppure lista vuota.
- ricerca in una lista di dizionari. Trovare il recoro con *una* chiave uguale (es. cognome) oppure con tutte le chiavi uguali (es. nome e cognome)

Ricerca binaria

Prerequisito fondamentale: la ricerca binaria funziona solo se la lista è **ordinata**.

Per esempio:

```
lista = [2, 3, 5, 7, 9, 12, 15]
```

Idea dell'algoritmo

La ricerca binaria sfrutta l'ordinamento della lista.

Invece di controllare gli elementi uno alla volta, confronta il valore cercato con l'elemento centrale della lista.

A ogni passo:

1. si considera la parte di lista in cui il valore potrebbe trovarsi;
2. si calcola la posizione centrale;
3. si confronta l'elemento centrale con il valore cercato;
4. se sono uguali, il valore è stato trovato;
5. se il valore cercato è più piccolo, si continua nella metà sinistra;
6. se il valore cercato è più grande, si continua nella metà destra.

A ogni iterazione, quindi, la quantità di elementi da controllare si dimezza.

Esempio

Supponiamo di avere questa lista ordinata:

```
lista = [2, 3, 5, 7, 9, 12, 15]
```

Vogliamo cercare il valore 12.

All'inizio consideriamo tutta la lista.

Passo	Intervallo considerato	Elemento centrale	Confronto
1	[2, 3, 5, 7, 9, 12, 15]	7	12 è maggiore di 7
2	[9, 12, 15]	12	trovato

Il valore 12 si trova in posizione 5.

Implementazione iterativa in Python

```
def ricerca_binaria(lista, valore):  
    sinistra = 0  
    destra = len(lista) - 1  
  
    while sinistra <= destra:  
        centro = (sinistra + destra) // 2
```

```

        if lista[centro] == valore:
            return centro
        elif valore < lista[centro]:
            destra = centro - 1
        else:
            sinistra = centro + 1

    return None

```

La funzione restituisce:

- l'indice dell'elemento, se il valore viene trovato;
- `None`, se il valore non è presente nella lista.

Esempio d'uso:

```

#| echo: true

lista = [2, 3, 5, 7, 9, 12, 15]

posizione = ricerca_binaria(lista, 12)
print(posizione)

```

Output: 5

Perché si usa `//`?

Nel codice:

```
centro = (sinistra + destra) // 2
```

l'operatore `//` indica la **divisione intera**.

Serve perché l'indice della lista deve essere un numero intero.

Per esempio:

```

(0 + 6) // 2  # risultato: 3
(4 + 5) // 2  # risultato: 4

```

Complessità computazionale della ricerca binaria

Indichiamo con n il numero di elementi della lista.

La ricerca binaria non controlla tutti gli elementi. A ogni passo dimezza lo spazio di ricerca.

- Dopo il primo confronto rimangono circa $\frac{n}{2}$ elementi
- Dopo il secondo confronto rimangono circa $\frac{n}{4}$ elementi
- Dopo il terzo confronto rimangono circa $\frac{n}{8}$ elementi
- ...
- Dopo il k -esimo confronto rimangono circa $\frac{n}{2^k}$ elementi

L'algoritmo termina quando rimane al massimo un elemento da controllare. Quindi cerchiamo il valore di k tale che:

$$\frac{n}{2^k} = 1$$

e quindi

$$k = \log_2(n)$$

Il numero di confronti cresce in modo logaritmico rispetto alla dimensione della lista. La complessità della ricerca binaria nel caso peggiore è:

$$O(\log n)$$

Quando si scrive $O(\log n)$, nella maggior parte dei contesti informatici si intende il logaritmo in base 2. La base del logaritmo non cambia la classe di complessità, perché logaritmi con basi diverse differiscono solo per una costante moltiplicativa.

Osservazione importante sul costo dell'ordinamento

La ricerca binaria è più efficiente della ricerca lineare, ma richiede una lista ordinata.

Se la lista non è già ordinata, bisogna prima ordinarla. L'ordinamento ha un costo computazionale proprio.

Quindi:

- se bisogna fare una sola ricerca su una lista non ordinata, può convenire la ricerca lineare;
- se bisogna fare molte ricerche sulla stessa lista, può convenire ordinare la lista una volta e poi usare la ricerca binaria.

Macchine a stati finiti (cenni)

Esercizio

Scrivere una funzione che sommi tutti i valori di una lista, tranne il valore 13 ed il valore immediatamente successivo (nella posizione in lista) a 13

esempio:

```
nums1 = [1, 13, 10, 1, 13, 13, 13, 10, 1, 13] # -> 3
nums2 = [1, 13, 10, 1, 13, 13, 13, 10, 1]      # -> 3
nums3 = [13, 10, 1, 13, 13, 13, 10, 1]        # -> 2
```

Ecco una possibile soluzione:

- leggi l'elemento `el` della lista
- se `el==13` oppure se il `precedente==13`, non sommarlo
- altrimenti sommalo.
- quando `el` è il primo elemento, imponiamo un `precedente` fittizio
- prima di leggere il successivo elemento, aggiorniamo la variabile `precedente`

```
def sum_skip_el(nums, el_to_skip):
    sum_val = 0
    last_el = None
    for el in nums:
        if el != el_to_skip and last_el != el_to_skip:
            sum_val += el
        last_el = el
    return sum_val
```

o equivalentemente

```
def sum_skip_el(nums, el_to_skip):
    sum_val = 0
    last_el = None
    for el in nums:
        if not(el == el_to_skip or last_el == el_to_skip):
            sum_val += el
        last_el = el
    return sum_val
```

Il limite di queste soluzioni è che sono soluzioni ‘ad hoc’. Per ogni problema simile (data una lista somma alcuni elementi e saltane altri a seconda di particolari condizioni) occorre trovare una nuova soluzione. Non è possibile generalizzare la soluzione già trovata.

Per esempio:

- 1) Data una lista di numeri, scrivi una funzione che calcoli la somma di tutti gli elementi escludendo quelli che si trovano tra il valore 3 e il valore 7 (inclusi), intesi come posizioni nella lista.

```
lista = [1, 2, 3, 99, 100, 7, 4, 5]
```

Gli elementi da saltare sono: 3, 99, 100, 7 (tra la prima occorrenza di 3 e la prima di 7)

- 2) In un gioco, un personaggio acquisisce un superpotere quando trova i tre amuleti **a**, **b**, **c** in questa esatta sequenza. Il personaggio può utilizzare questo superpotere tre volte prima di perdere gli amuleti e ricominciare il processo.

Le seguenti sequenze di oggetti invece **non** consentono di acquisire il superpotere

- **a, b, x, c** (un altro oggetto interrompe la sequenza)
- **a, c, b** (un altro oggetto interrompe la sequenza)

Possiamo definire molte varianti di questo problema:

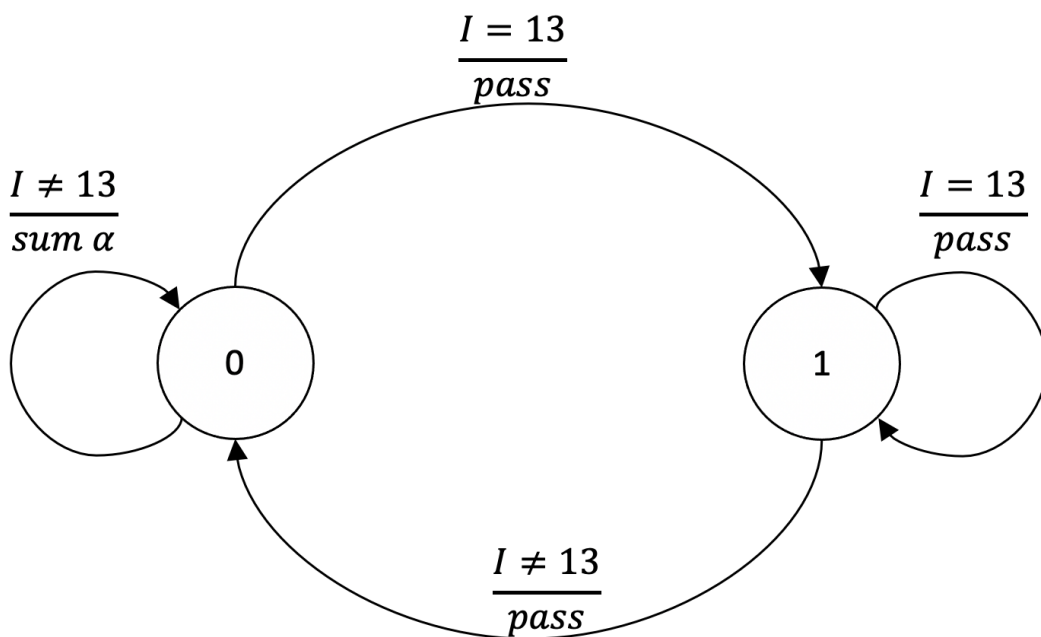
- se la sequenza viene interrotta, occorre ricominciare da capo nel percorso verso i superpoteri (oppure no)
- accettiamo sequenze anche interrotte da altri oggetti
- accettiamo sequenze anche in ordine diverso
- ...

In questi problemi è evidente un **modello comune**. Incontriamo un oggetto con uno **stato**, il cui comportamento è definito sia dall’input che dallo stato corrente. L’input innesca una transizione di stato.

Possiamo esaminare un approccio comune per affrontare questa categoria di problemi.

Una macchina a stati finiti ¹ (FSM) è una macchina astratta che può trovarsi in uno qualsiasi di un numero finito di stati. La FSM può passare da uno stato all'altro (transizione) in risposta a determinati input.

In una macchina di Mealy i valori di output sono determinati sia dal suo stato attuale che dagli input correnti. In una macchina di Moore i valori di output sono determinati esclusivamente dal suo stato attuale. È anche possibile associare azioni a uno stato: un'**azione di ingresso**, eseguita quando si entra nello stato, e un'**azione di uscita**, eseguita quando si esce dallo stato. <>



Possiamo descrivere l'esercizio `sum_skip` (che salta il valore 13 e l'elemento successivo) utilizzando il concetto di *stato*:

- L'oggetto ha due stati, 0 e 1.
- Nello stato 0, se l'input è $\neq 13$, l'oggetto somma i valori in input e rimane nello stato 0. Se l'input è 13, il valore viene saltato e l'oggetto passa allo stato 1.
- Nello stato 1, l'oggetto non somma i valori in input. Se l'input è 13, l'oggetto rimane nello stato 1. Se l'input è diverso da 13, l'oggetto passa allo stato 0 (e non stampa il valore).

¹https://en.wikipedia.org/wiki/Finite-state_machine

Soluzione:

```
def sum_skip(valori, el_to_skip):
    stato = 'somma'
    s = 0
    for el in valori:
        if stato == 'somma':
            if el == el_to_skip:
                stato = 'non_somma'
            else: # vuol dire che el != el_to_skip
                s = s+el
        else: # vuol dire che sono nello stato 'non_somma'
            if el == el_to_skip:
                pass
                #non faccio nulla e resto nello stato 'non_somma'
            else: #el != el_to_skip
                stato = 'somma'
    return s

nums1 = [1, 13, 10, 1, 13, 13, 13, 10, 1, 13] # -> 3
nums2 = [1, 13, 10, 1, 13, 13, 13, 10, 1]      # -> 3
nums3 = [13, 10, 1, 13, 13, 13, 10, 1]        # -> 2

print(sum_skip(nums3, 13) )
```
